



GROUP 5: DESIGN AND IMPLEMENTATION OF A JAVA DSA TOOLKIT

Our Lecturer:
Leng Rathana

Semester: **4**

Prepared by :

Chui Vanenteakbot - ID: 601007443

Keo Ratanak - ID: 601007563

Lao David - ID: 601007476

Sorn Kunwath - ID: 601007865

Top Keitaro - ID: 601007735

Faculty of Digital Innovation and Technology

BIT3253 - DATA STRUCTURE & ALGORITHM ANALYSIS

12 September 2026

Table of content

1. Introduction.....	3
2. Objectives.....	3
3. Group member details.....	3
4. Explanation of Implemented Data Structures.....	4
4.1. Dynamic Array.....	4
4.2. Singly Linked List.....	4
4.3. Stack.....	5
4.4. Queue.....	5
4.5. Binary Search Tree.....	6
5. Explanation of Implemented Algorithms.....	6
5.1. Searching Algorithms.....	6
5.1.1. Linear Search.....	6
5.1.2. Binary Search.....	6
5.2. Sorting Algorithms.....	7
5.2.1. Bubble Sort.....	7
5.2.2. Selection Sort.....	7
5.2.3. Insertion Sort.....	7
5.2.4. Merge Sort.....	8
5.2.5. Quick Sort.....	8
5.2.6. Heap Sort.....	8
6. Big O Complexity Analysis.....	9
6.1. Data Structure Complexity.....	9
6.2. Sorting Algorithm Complexity.....	9
6.3. Searching Algorithm Complexity.....	10
6.4. Overall Analysis.....	10
7. Demonstration of Correctness with Sample Runs.....	10
7.1. Data Structure Testing.....	10
7.2. Searching Algorithms Testing.....	11
7.3. Sorting Algorithms Testing.....	12
8. Benchmarking Results and Discussion.....	13
8.1. Benchmarking Methodology.....	13
8.2. Sorting Algorithm Benchmark Results.....	13
8.3. Searching Algorithm Benchmark Results.....	13
8.4. Data Structure Benchmark Results.....	14
8.5. Benchmark Discussion.....	15
9. Comparison of Alternatives and Trade-offs.....	15
9.1. Data Structure Comparisons.....	15
9.2. Algorithm Comparisons.....	16
10. Conclusion.....	16
11. Contribution Statement.....	17

1. Introduction

Data structures and algorithms are important foundations of software development because they determine how data is stored, accessed, processed, and manipulated within a program. Different data structures are suitable for different types of problems, while different algorithms can provide different levels of efficiency when processing data.

This project "Design and Implementation of a Java DSA Toolkit" focuses on designing and implementing a collection of fundamental data structures and algorithms using Java. The toolkit is developed from scratch to demonstrate how common data structures operate internally rather than relying on Java's built-in collection classes for the core implementations.

The implemented data structures include a Dynamic Array, Singly Linked List, Stack, Queue, and Binary Search Tree. The project also covers fundamental algorithms, including Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Linear Search, and Binary Search.

In addition to implementing these structures and algorithms, the project evaluates their efficiency using Big O notation. Time and space complexity are considered, including best-case, average-case, and worst-case scenarios where appropriate. The project also includes benchmarking to measure the practical execution time of selected algorithms and data structures using different input sizes. These measured results can then be compared with their theoretical complexity.

Finally, the toolkit includes demonstrations and test cases to verify that the implemented structures and algorithms produce correct results. Normal cases and relevant edge cases are considered to provide evidence that the implementations work correctly and reliably.

2. Objectives

The main objectives of this project are:

1. To design and implement fundamental data structures in Java from scratch, including Dynamic Array, Singly Linked List, Stack, Queue, and Binary Search Tree.
2. To implement fundamental searching and sorting algorithms, including Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.
3. To understand and analyze algorithm efficiency using Big O notation, including time and space complexity as well as best-case, average-case, and worst-case performance where applicable.
4. To test the correctness of the implemented data structures and algorithms using normal inputs and appropriate edge cases such as empty input, single-element input, duplicate values, and already sorted data.
5. To benchmark selected algorithms and data structures using multiple input sizes and repeated measurements in order to observe their practical performance.
6. To compare different data structures and algorithms based on their advantages, disadvantages, efficiency, and suitable use cases.
7. To apply object-oriented programming principles in Java by organizing the toolkit into clear classes and methods with meaningful names and maintainable code.
8. To develop a working Java DSA toolkit and demonstration program that shows the implementation, usage, and performance characteristics of the selected data structures and algorithms.

3. Group member details

No.	Name	Responsibility
1	Top Keitaro	Singly Linked List and Insertion Sort
2	Lao David	Binary Search Tree and Binary Search
3	Sorn Kunwath	Stack and Selection Sort
4	Chui Vanenteakbot	Queue and Bubble Sort
5	Keo Ratanak	Dynamic Array and Linear Search

Shared Responsibility: All group members contributed to the implementation and testing of Merge Sort, Quick Sort, and Heap Sort.

Contribution: Each group member was responsible for implementing and testing their assigned data structures and algorithms. The group also worked together on debugging, integration, correctness testing, benchmarking, documentation, and preparation of the final presentation.

4. Explanation of Implemented Data Structures

4.1. Dynamic Array

A Dynamic Array is a linear data structure that can automatically expand its capacity when its current storage becomes full. It stores elements in a continuous array and allows the number of stored elements to grow during program execution.

In this project, the Dynamic Array is implemented from scratch in Java using an internal integer array. The implementation keeps track of the current number of elements and the allocated capacity. When the array becomes full, a larger array is created and the existing elements are copied into it.

The main operations implemented are:

- Add: Adds an element to the end of the dynamic array.
- Add At: Inserts an element at a specific position and shifts the following elements.
- Get: Returns the element at a specified position.
- Set: Updates the value at a specific position.
- Find: Searches for an element and returns its position.
- Contains: Checks if a specific element exists in the array.
- Remove: Removes an element from a specific position and shifts the remaining elements.
- Is Empty: Checks whether the dynamic array contains any elements.
- Size: Returns the current number of elements stored.
- Capacity: Return the current allocated capacity.
- Display: Displays the elements currently stored in the dynamic array.

Complexity Analysis: Accessing an element by index takes $O(1)$ time. Searching for an element takes $O(n)$. Adding an element at the end is $O(1)$ when capacity is available, but can take $O(n)$ when the array needs to be resized. Inserting or removing an element at a specific position can take $O(n)$ because elements may need to be shifted. The space complexity is $O(n)$.

Advantages and Suitable Use: When more elements are added, a dynamic array can dynamically expand its capacity and offer quick access to elements by index. It is appropriate for applications that require fast access to data by position and when the number of items varies regularly.

4.2. Singly Linked List

A Singly Linked List is a linear data structure made up of nodes. Each node stores a data value and a reference to the next node. Unlike an array, the elements are not stored next to each other in memory. Instead, each node connects to the next node through its reference.

In this project, the Singly Linked List is implemented from scratch in Java using a separate Node class. The list keeps a reference to the first node, called head, and supports adding, removing, searching, updating, and displaying elements.

The main operations implemented are:

- Insert First: Adds an element at the beginning of the list.
- Insert Last: Adds an element at the end of the list.
- Insert At: Adds an element at a specific position.
- Delete First: Removes the first element.
- Delete Last: Removes the last element.
- Delete At: Removes an element from a specific position.
- Search: Searches for an element in the list.
- Update: Changes the value of an existing element.

- **Display:** Displays the elements in the list.

Complexity Analysis: Inserting or deleting at the beginning takes $O(1)$ because the head reference can be updated directly. Operations such as `insertLast`, `deleteLast`, `insertAt`, `deleteAt`, `search`, and `update` can require traversal and therefore take $O(n)$ time in the worst case. The list uses $O(n)$ space because each element is stored in a separate node.

Advantages and Suitable Use: A Singly Linked List allows elements to be added or removed without shifting other elements. This makes it useful when frequent insertions and deletions are needed. However, elements cannot be accessed directly by index, so finding a specific position requires starting from the head and moving through the nodes.

4.3. Stack

A Stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. This means that the last element added to the stack is the first element removed. Elements are added and removed from the top of the stack.

In this project, the Stack is implemented from scratch in Java using an array. The implementation maintains the current top position and provides operations for adding, removing, and viewing elements.

The main operations implemented are:

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes and returns the top element.
- **Peek:** Returns the top element without removing it.
- **Is Empty:** Checks whether the stack contains no elements.
- **Is Full:** Checks whether the array-based stack has reached its capacity.
- **Size:** Returns the number of elements in the stack.
- **Display:** Shows the elements stored in the stack.

Complexity Analysis: The push, pop, peek, `isEmpty`, `isFull`, and size operations take $O(1)$ time because they directly access or update the top position. The display operation takes $O(n)$ time because it goes through all elements in the stack. The array-based Stack uses $O(n)$ space based on its allocated capacity.

Advantages and Suitable Use: A Stack provides efficient insertion and removal from one end and is suitable for applications such as function calls, undo operations, expression processing, and backtracking. The array-based implementation is simple and provides fast access to the top element, but it has a fixed capacity.

4.4. Queue

A Queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. This means that the first element added to the queue is the first element removed. Elements are added from the rear and removed from the front.

In this project, the Queue is implemented from scratch in Java and includes both an array-based queue and a linked queue. The array-based queue uses a circular structure, allowing positions that become available after dequeue operations to be reused without shifting the remaining elements. The linked queue uses nodes and maintains references to the front and rear of the queue.

The main operations implemented are:

- **Enqueue:** Adds an element to the rear of the queue.
- **Dequeue:** Removes and returns the element at the front.
- **Peek:** Returns the front element without removing it.
- **Is Empty:** Checks whether the queue contains any elements.

The implementation also handles empty queue and full queue conditions. The circular array allows the array-based queue to reuse available positions, while the linked queue can grow dynamically as new nodes are added.

Complexity Analysis: For both the array-based and linked queue, enqueue, dequeue, peek, and `isEmpty` operate in $O(1)$ time because the implementation directly accesses the required front or rear

positions without traversing all elements. Both implementations require $O(n)$ space to store the queue elements.

Advantages and Suitable Use: The array-based queue provides efficient storage with a fixed capacity, while the linked queue provides greater flexibility because its size can grow dynamically. Both are suitable for situations where elements must be processed in their arrival order, such as task processing, printer scheduling, and request handling.

4.5. Binary Search Tree

A Binary Search Tree (BST) is a hierarchical data structure where each node has at most two children. Values smaller than a node are stored in the left subtree, while larger values are stored in the right subtree. This arrangement allows data to be searched efficiently when the tree is balanced.

In this project, the Binary Search Tree is implemented from scratch in Java using nodes with references to their left and right children. The tree maintains a root node and supports inserting, searching, deleting, finding minimum and maximum values, and tree traversal.

The main operations implemented are:

- Insert: Adds a value to its correct position.
- Search: Searches for a specified value.
- Delete: Removes a value while maintaining the BST structure.
- Find Minimum / Maximum: Finds the smallest or largest value.
- Traversal: Includes in-order, pre-order, and post-order traversal.

Complexity Analysis: Search, insertion, and deletion take $O(\log n)$ on average when the tree is reasonably balanced, but can become $O(n)$ in the worst case when the tree is unbalanced. Traversals take $O(n)$ because every node must be visited. The space complexity is $O(n)$.

Advantages and Suitable Use: A BST supports efficient searching, insertion, and deletion while maintaining data in an ordered structure. However, an unbalanced tree can reduce performance to $O(n)$.

5. Explanation of Implemented Algorithms

5.1. Searching Algorithms

5.1.1. Linear Search

Linear Search is a simple searching algorithm that checks each element in a list step by step from the beginning until the target element is located or the list's end is reached.

In this project, Linear Search is implemented from scratch in Java using an integer array. The method compares each element with the target value from the first element to the last. The element's index is returned if the target is located. If the target is not found after checking all elements, the algorithm returns -1.

For example, given the array [10, 25, 40, 15, 30] and the target value 30, the algorithm checks each element from left to right. When it reaches 30 at index 4, the method returns 4.

Complexity Analysis: When the target is located at the first position, the best-case time complexity is $O(1)$. Because the method may need to check many or all elements, the average-case and worst-case time complexities are $O(n)$. Since no extra data structure is needed, the space complexity is $O(1)$.

Advantages and Suitable Use: It is easy to understand and doesn't require sorting the data before searching. And it works well with unsorted or tiny datasets where search performance is not as critical as simplicity.

5.1.2. Binary Search

Binary Search is a searching algorithm that repeatedly divides a **sorted** array into smaller sections. It compares the target with the middle element and continues searching in either the left or right half depending on the comparison.

In this project, Binary Search is implemented from scratch in Java using an integer array sorted in ascending order. The implementation uses left and right boundaries to determine the current search range. If the middle value matches the target, its index is returned. Otherwise, the search continues in the appropriate half.

The implementation also handles duplicate values by continuing to search toward the left to return the first matching index. It also handles empty arrays and null input.

For example, given the sorted array [1, 3, 5, 7, 9] and target 7, the algorithm checks the middle values and eventually finds 7 at index 3. If the target is not found, it returns -1.

Complexity Analysis: The best-case time complexity is $O(1)$ when the target is found at the middle position. The average-case and worst-case time complexities are $O(\log n)$ because the search range is approximately halved after each comparison. The space complexity is $O(1)$.

Advantages and Suitable Use: Binary Search is much faster than Linear Search for large sorted datasets. However, the data must be sorted before the algorithm can be used.

5.2. Sorting Algorithms

5.2.1. Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. For ascending order, if the left element is greater than the right element, they are swapped. After each pass, the largest unsorted element moves to its correct position at the end of the array.

In this project, Bubble Sort is implemented from scratch in Java using nested loops. The outer loop controls the number of passes, while the inner loop compares adjacent elements and performs the swap when necessary. The process continues until all elements are arranged in ascending order.

For example, given the array [5, 3, 8, 1], the algorithm compares 5 and 3 and swaps them, then moves forward to compare the next adjacent elements. After the first complete pass, the largest value (8) is correctly placed at the final index of the array.

Complexity Analysis: The best-case, average-case, and worst-case time complexities are $O(n^2)$ for this implementation because the nested loops continue comparing the elements even when the array is already sorted. The space complexity is $O(1)$ because the algorithm sorts the array in-place and only requires a temporary variable for swapping elements.

Advantages and Suitable Use: Bubble Sort is intuitive and simple to implement, making it useful for educational purposes and small datasets. However, its quadratic time complexity makes it inefficient for large datasets compared to algorithms such as Merge Sort and Quick Sort.

5.2.2. Selection Sort

Selection Sort is a sorting algorithm that repeatedly finds the smallest element from the unsorted part of an array and places it at the beginning of that part. The process continues until all elements are arranged in ascending order.

In this project, Selection Sort is implemented from scratch in Java using nested loops. The outer loop determines the position where the next smallest element should be placed, while the inner loop searches the remaining unsorted elements to find the smallest value. Once the smallest value is found, it is swapped with the element at the current position.

For example, given the array [5, 3, 8, 1], the algorithm first finds 1 as the smallest value and swaps it with 5, resulting in [1, 3, 8, 5]. It then searches the remaining unsorted elements and continues the same process until the array is sorted.

Complexity Analysis: The best-case, average-case, and worst-case time complexities are $O(n^2)$ because the algorithm still searches through the remaining unsorted elements on every pass, even when the array is already sorted. The space complexity is $O(1)$ because the sorting is performed in-place and only a temporary variable is needed for swapping.

Advantages and Suitable Use: Selection Sort is simple to understand and uses very little additional memory, making it suitable for small datasets and educational purposes. However, its $O(n^2)$ time complexity makes it inefficient for large datasets compared with more efficient algorithms such as Merge Sort, Quick Sort, and Heap Sort.

5.2.3. Insertion Sort

Insertion Sort is a sorting algorithm that builds a sorted portion of an array one element at a time. It starts from the second element and compares it with the elements before it. If the previous elements are

greater, they are shifted one position to the right until the current element can be placed in its correct position.

In this project, Insertion Sort is implemented from scratch in Java. The implementation uses a key variable to store the current element and a j variable to move through the sorted portion of the array. Elements greater than the key are shifted to the right, and the key is then inserted into its correct position.

For example, given the array [5, 2, 4, 6, 1, 3], the algorithm starts with 2 and inserts it before 5, producing [2, 5, 4, 6, 1, 3]. It then processes 4, 6, 1, and 3 in the same way until the array becomes [1, 2, 3, 4, 5, 6].

Complexity Analysis: The best-case time complexity is $O(n)$ when the array is already sorted because each element only needs to be compared with the previous element. The average-case and worst-case time complexities are $O(n^2)$ because elements may need to be shifted multiple positions. The space complexity is $O(1)$ because the algorithm sorts the array in-place and only uses a small number of additional variables.

Advantages and Suitable Use: Insertion Sort is simple to understand and implement and performs well on small or nearly sorted datasets. It also sorts the array in-place without requiring additional memory. However, its $O(n^2)$ average and worst-case time complexity makes it inefficient for large or highly unsorted datasets compared with algorithms such as Merge Sort and Heap Sort.

5.2.4. Merge Sort

Merge Sort is a sorting algorithm that divides an array into smaller sub-arrays until each part contains one element. It then merges the sub-arrays while comparing their elements to produce a sorted array.

In this project, Merge Sort is implemented from scratch in Java using a recursive divide-and-conquer approach. The array is divided into two halves, each half is sorted recursively, and the sorted halves are then merged together.

For example, given the array [5, 3, 8, 1], the algorithm divides it into smaller parts and merges them in sorted order until the final array becomes [1, 3, 5, 8].

Complexity Analysis: The best-case, average-case, and worst-case time complexities are $O(n \log n)$ because the array is repeatedly divided and the elements are merged at each level. The space complexity is $O(n)$ because an additional temporary array is used during the merging process.

Advantages and Suitable Use: Merge Sort provides consistent $O(n \log n)$ performance and is suitable for large datasets. However, it requires additional memory compared with in-place sorting algorithms.

5.2.5. Quick Sort

Quick Sort is a sorting algorithm that selects a pivot element and partitions the array into smaller and larger elements around the pivot. The same process is then applied recursively to the resulting sections.

In this project, Quick Sort is implemented from scratch in Java using a pivot-based partitioning approach. The implementation uses the middle element as the pivot before partitioning the array, then recursively sorts the resulting parts.

For example, given the array [5, 3, 8, 1], the algorithm selects a pivot and rearranges the elements around it. The partitions are then sorted recursively until the array becomes [1, 3, 5, 8].

Complexity Analysis: The best-case and average-case time complexities are $O(n \log n)$ when the partitions are reasonably balanced. The worst-case time complexity is $O(n^2)$ when the partitions become highly unbalanced. The auxiliary space complexity is $O(\log n)$ on average due to recursive calls, but can become $O(n)$ in the worst case.

Advantages and Suitable Use: Quick Sort is efficient for large datasets and can sort elements in-place with low additional memory usage. However, its performance depends on the partitioning, and poorly balanced partitions can result in worse performance.

5.2.6. Heap Sort

Heap Sort is a sorting algorithm that uses a binary heap to repeatedly select the largest element and place it at the end of the array. The remaining elements are then reorganized to maintain the heap structure until the array is sorted.

In this project, Heap Sort is implemented from scratch in Java using a max heap. The implementation first builds the max heap and then repeatedly moves the largest element to its correct position before restoring the heap structure.

For example, given the array [5, 3, 8, 1], the algorithm builds a max heap, moves the largest value to the end, and continues this process until the array becomes [1, 3, 5, 8].

Complexity Analysis: The best-case, average-case, and worst-case time complexities are $O(n \log n)$ because the heap requires logarithmic adjustments when elements are reorganized. The space complexity is $O(1)$ because the sorting is performed in-place.

Advantages and Suitable Use: Heap Sort provides consistent $O(n \log n)$ performance and requires very little additional memory. However, it is more complex to understand and implement compared with simpler sorting algorithms.

6. Big O Complexity Analysis

Big O notation is used to analyze the efficiency of the data structures and algorithms implemented in this project. It describes how the running time and memory requirements change as the input size, represented by n , increases. The analysis considers the best-case, average-case, and worst-case time complexity, as well as the additional space required by each implementation.

6.1. Data Structure Complexity

Data Structure	Typical Operation	Best Case	Average Case	Worst Case	Space
Dynamic Array	Access / Search / Insert / Delete	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	Insert / Delete / Search	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	Push / Pop / Peek	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Queue	Enqueue / Dequeue / Peek	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Binary Search Tree	Search / Insert / Delete	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$

The data structures have different performance characteristics depending on the operation being performed. Stack and Queue operations are $O(1)$ because they directly access the required end of the structure. A Singly Linked List can perform operations at the beginning in $O(1)$, while operations that require traversal can take $O(n)$. A Dynamic Array provides $O(1)$ direct access, but insertion or deletion may require shifting elements. A Binary Search Tree can achieve $O(\log n)$ when the tree is reasonably balanced, while its worst-case performance becomes $O(n)$ when the tree is unbalanced.

6.2. Sorting Algorithm Complexity

Sorting Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ average / $O(n)$ worst
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

The sorting algorithms show clear differences in efficiency. Bubble Sort and Selection Sort have $O(n^2)$ time complexity, making them less suitable for large datasets. Insertion Sort has a best-case complexity of $O(n)$ when the input is already sorted, although its average and worst cases remain $O(n^2)$. Merge Sort and Heap Sort provide $O(n \log n)$ worst-case performance, while Quick Sort normally performs in $O(n \log n)$ but can degrade to $O(n^2)$ in the worst case.

6.3. Searching Algorithm Complexity

Searching Algorithm	Best Case	Average Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Linear Search can find the target immediately when it is at the first position, giving $O(1)$ best-case performance. However, it may need to examine every element, resulting in $O(n)$ average and worst-case performance. Binary Search is more efficient for sorted data because it reduces the search range by approximately half at each step, resulting in $O(\log n)$ average and worst-case performance.

6.4. Overall Analysis

Overall, the complexity analysis shows that different data structures and algorithms are suitable for different situations. $O(n^2)$ algorithms such as Bubble Sort and Selection Sort are simple to implement but become less efficient as the input size increases. Algorithms such as Merge Sort, Quick Sort, and Heap Sort generally provide better scalability for larger datasets. For searching, Binary Search provides better performance than Linear Search when the data is sorted.

Space complexity is also an important consideration. Bubble Sort, Selection Sort, Insertion Sort, and Heap Sort use $O(1)$ additional space, while Merge Sort requires $O(n)$ additional space. Therefore, both execution time and memory usage should be considered when selecting an appropriate data structure or algorithm.

Exponential complexity such as $O(2^n)$ is not present among the implemented algorithms in this toolkit, but it represents a much faster growth rate than the polynomial-time algorithms considered in this project.

7. Demonstration of Correctness with Sample Runs

This section demonstrates the correctness of the implemented data structures and algorithms through sample test runs. Normal cases and relevant edge cases, including empty input, single-element input, duplicate values, and already sorted data, are tested where appropriate. The expected and actual results are compared to verify that the implementations produce the correct output.

7.1. Data Structure Testing

Data Structure	Main Operations Tested	Edge Cases	Result
Dynamic Array	Add, Add At, Get, Set, Remove	Empty, single element	PASS
Singly Linked List	Insert, Delete, Search, Update	Empty, single element	PASS
Stack	Push, Pop, Peek, Size	Empty, full capacity	PASS
Queue	Enqueue, Dequeue, Peek	Empty, single element, full queue	PASS
Binary Search Tree	Insert, Search, Delete, Traversal	Empty, duplicates, deletion cases	PASS

The data structure tests confirm that the implemented operations produce the expected results. Boundary and edge cases are also tested to verify that the structures handle different input conditions correctly.

```
Pushing items...
Stack (top -> bottom): 30 20 10
Stack size: 3
Top item: 30
Is stack full? false

Popping item...
Removed item: 30
Stack (top -> bottom): 20 10
Stack size: 2

Pushing more items...
Stack (top -> bottom): 60 50 40 20 10
Stack size: 5
Is stack full? true

Popping items...
Removed: 60
Removed: 50
Stack (top -> bottom): 40 20 10

Is stack empty? false
Figure 7.1. Stack Test Results
```

```
=== Linked Queue ===
Peek: 10
Dequeue: 10
Peek after Dequeue: 20
Is Empty: false

=== Array Queue ===
Peek: 10
Is Full: true
Dequeue: 10
Peek after Circular Enqueue: 20
Is Empty: false

Linked Queue Empty Test: Queue Underflow
Array Queue Full Test: Queue Overflow
Figure 7.2. Queue Test Results
```

```
===== BST TEST =====

--- Test 1: Insert ---
In-Order: 20 30 40 50 60 70 80

--- Test 2: Search ---
Search 40: true
Search 120: false

--- Test 3: Traversals ---
Pre-Order: 50 30 20 40 70 60 80
Post-Order: 20 40 30 60 80 70 50

--- Test 4: Min / Max ---
Minimum: 20
Maximum: 80

--- Test 5: Duplicate ---
After inserting duplicate 40: 20 30 40 50 60 70 80

--- Test 6: Delete Leaf ---
After deleting 20: 30 40 50 60 70 80

--- Test 7: Delete One Child ---
After deleting 30: 40 50 60 70 80

--- Test 8: Delete Two Children ---
After deleting 70: 40 50 60 80

--- Test 9: Delete Root ---
After deleting root 50: 40 60 80

--- Test 10: Empty Tree ---
Empty Tree Min Test: Tree is Empty!
Empty Tree Max Test: Tree is Empty!
Figure 7.3. Binary Search Tree Test Results
```

Figure 7.5. Singly Linked List Test Results

```

===== Normal Cases =====
--- Insert Test ---
Line: Head -> 30 -> 10 -> 20 -> 50 -> null
-----

--- Insert At Test ---
After insert:
Line: Head -> 30 -> 10 -> 25 -> 20 -> 50 -> null
-----

--- Delete Test ---
After deleteFirst():
Line: Head -> 10 -> 25 -> 20 -> 50 -> null
After deleteLast():
Line: Head -> 10 -> 25 -> 20 -> null
After deleteAt():
Line: Head -> 10 -> 20 -> null
-----

--- Search Test ---
Search 25: false
Search 20: true
-----

--- Update Test ---
After updated:
Line: Head -> 98 -> 20 -> null
-----

===== Edge Cases =====

--- Empty list ---
Line: Head -> null
After deleting from empty list:
Line: Head -> null
-----

--- One element ---
Line: Head -> 100 -> null
After deleting the only element:
Line: Head -> null
-----

--- Duplicate values ---
Line: Head -> 10 -> 20 -> 20 -> 30 -> null
Search 20: true
-----

--- Insert at position 0 ---
Line: Head -> 10 -> 20 -> 30 -> null
-----

--- Insert at last position ---
Line: Head -> 10 -> 20 -> 30 -> 40 -> null
-----

--- Delete the only element ---
Before deleting only element:
Line: Head -> 50 -> null
After deleting only element:
Line: Head -> null
-----

--- Search for a value that not exist ---
Search missing value:
Search 99: false
-----

--- Invalid position test ---
Caught invalid position.
-----

```

Figure 7.4. Dynamic Array Test Results

```

===== DYNAMIC ARRAY TEST =====

--- Test 1: Empty Array ---
Is empty: true
Size: 0

--- Test 2: One Element ---
Dynamic Array: [99]
Size: 1
First value: 99

--- Test 3: Add Elements ---
Dynamic Array: [10, 20, 30]

--- Test 4: Automatic Resizing ---
Before resizing:
Dynamic Array: [10, 20, 30, 40]
Capacity: 4
After adding 50:
Dynamic Array: [10, 20, 30, 40, 50]
Capacity after resizing: 6

--- Test 5: Insert at Position ---
After inserting 15 at position 1:
Dynamic Array: [10, 15, 20, 30, 40, 50]

--- Test 6: Update ---
After updating position 2 to 25:
Dynamic Array: [10, 15, 25, 30, 40, 50]

--- Test 7: Get / Access ---
Value at position 3: 30

--- Test 8: Search ---
30 found at position: 3
100 not found.

--- Test 9: Contains ---
Contains 25: true
Contains 100: false

--- Test 10: Delete ---
Removed value: 15
After deletion:
Dynamic Array: [10, 25, 30, 40, 50]

--- Test 11: Duplicate Values ---
Dynamic Array: [10, 25, 30, 40, 50, 30, 30]

--- Final Information ---
Final size: 7
Final capacity: 9

===== DYNAMIC ARRAY TEST COMPLETE =====

```

7.2. Searching Algorithms Testing

Algorithm	Main Tests	Edge Cases	Result
Linear Search	Target found, target not found	Empty, single element, duplicates, sorted array	PASS
Binary Search	Target found, target not found	Empty, single element, duplicates, sorted array	PASS

The searching algorithms were tested using normal and relevant edge-case inputs to verify that they produce the expected results. Linear Search was tested with targets that are present and absent, including empty arrays, single-element arrays, duplicate values, and sorted input. Binary Search was tested using sorted arrays with targets that are present and absent, including duplicate values, empty arrays, and single-element arrays. The test results are compared with the expected behaviour to confirm the correctness of both searching algorithms.

```
===== BINARY SEARCH TEST =====
```

```

--- Normal Search ---
Expected: 3, Actual: 3

--- Target at Beginning ---
Expected: 0, Actual: 0

--- Target at Middle ---
Expected: 2, Actual: 2

--- Target at End ---
Expected: 4, Actual: 4

--- Target Not Found ---
Expected: -1, Actual: -1

--- Empty Array ---
Expected: -1, Actual: -1

--- One Element Found ---
Expected: 0, Actual: 0

--- One Element Not Found ---
Expected: -1, Actual: -1

--- Duplicates ---
Expected first index: 1, Actual: 1

--- Already-Sorted Input ---
Expected: 3, Actual: 3

--- Null Input ---
Null input throws IllegalArgumentException.

```

Figure 7.7. Binary Search Test Results

```
===== LINEAR SEARCH DEMO =====
```

```

--- Test 1: Normal Case ---
Array: [10, 25, 40, 15, 30]
Search value: 30
Result index: 4

--- Test 2: Value Not Found ---
Array: [10, 25, 40, 15, 30]
Search value: 100
Result index: -1

--- Test 3: Empty Array ---
Array: []
Search value: 10
Result index: -1

--- Test 4: One Element ---
Array: [50]
Search value: 50
Result index: 0

--- Test 5: Duplicate Values ---
Array: [10, 20, 20, 30, 20]
Search value: 20
First matching index: 1

--- Test 6: Already Sorted Array ---
Array: [10, 20, 30, 40, 50]
Search value: 40
Result index: 3

===== DEMO COMPLETE =====

```

Figure 7.6. Linear Search Test Results

7.3. Sorting Algorithms Testing

Algorithm	Main Tests	Edge Cases	Result
Bubble Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS
Selection Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS
Insertion Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS
Merge Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS
Quick Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS
Heap Sort	Unsorted array, sorted output	Empty, single element, duplicates, sorted array	PASS

The sorting algorithms were tested using normal and relevant edge-case inputs to verify that each algorithm produces the expected sorted result. The tests include unsorted arrays, already sorted arrays, empty arrays, single-element arrays, and duplicate values. The output of each algorithm was compared with the expected sorted order to confirm that all six sorting implementations work correctly.

Normal array:
3 4 5 6 8
Empty array:
5
One element:
3 3 5 5
Already sorted:
1 2 3 4

Normal before sorting: [27, 36, 45, 1, 23, 28, 98]
Normal after sorting: [1, 23, 27, 28, 36, 45, 98]
Empty before sorting: []
Empty after sorting: []
One before sorting: [11]
One after sorting: [11]
Duplicates before sorting: [12, 28, 11, 12, 23, 11]
Duplicates after sorting: [11, 11, 12, 12, 23, 28]
Sorted before sorting: [1, 2, 3, 4, 5, 6]
Sorted after sorting: [1, 2, 3, 4, 5, 6]

Figure 7.8. Bubble Sort Test Results

Figure 7.9. Selection Sort Test Results

```
=====
INSERTION SORT TESTING
=====
Test 1 - Normal Unsorted Array
Input: [5, 2, 4, 6, 1, 3]
Expected: [1, 2, 3, 4, 5, 6]
Actual: [1, 2, 3, 4, 5, 6]
Result: PASS

Test 2 - Empty Array
Input: []
Expected: []
Actual: []
Result: PASS

Test 3 - One Element
Input: [10]
Expected: [10]
Actual: [10]
Result: PASS

Test 4 - Duplicate Values
Input: [5, 3, 5, 2, 3, 1, 5]
Expected: [1, 2, 3, 3, 5, 5, 5]
Actual: [1, 2, 3, 3, 5, 5, 5]
Result: PASS

Test 5 - Already Sorted Array
Input: [1, 2, 3, 4, 5, 6]
Expected: [1, 2, 3, 4, 5, 6]
Actual: [1, 2, 3, 4, 5, 6]
Result: PASS

Test 6 - Reverse Sorted Array
Input: [6, 5, 4, 3, 2, 1]
Expected: [1, 2, 3, 4, 5, 6]
Actual: [1, 2, 3, 4, 5, 6]
Result: PASS

Test 7 - Negative and Positive Values
Input: [3, -2, 7, -5, 0, 4, -1]
Expected: [-5, -2, -1, 0, 3, 4, 7]
Actual: [-5, -2, -1, 0, 3, 4, 7]
Result: PASS

=====
ALL TESTS COMPLETED
=====
```

Figure 7.10. Insertion Sort Test Results

```
===== HEAP SORT TEST =====

--- Normal Case ---
Before: [8, 5, 3, 6, 4]
After: [3, 4, 5, 6, 8]
Expected: [3, 4, 5, 6, 8]
Result: PASS

--- Empty Array ---
Before: []
After: []
Expected: []
Result: PASS

--- One Element ---
Before: [11]
After: [11]
Expected: [11]
Result: PASS

--- Duplicates ---
Before: [5, 3, 5, 3, 1, 5]
After: [1, 3, 3, 5, 5, 5]
Expected: [1, 3, 3, 5, 5, 5]
Result: PASS

--- Already Sorted ---
Before: [1, 2, 3, 4, 5]
After: [1, 2, 3, 4, 5]
Expected: [1, 2, 3, 4, 5]
Result: PASS
```

Figure 7.13. Heap Sort Test Results

```
===== QUICK SORT TEST =====

--- Normal Case ---
Before: [8, 5, 3, 6, 4]
After: [3, 4, 5, 6, 8]
Expected: [3, 4, 5, 6, 8]
Result: PASS

--- Empty Array ---
Before: []
After: []
Expected: []
Result: PASS

--- One Element ---
Before: [11]
After: [11]
Expected: [11]
Result: PASS

--- Duplicates ---
Before: [5, 3, 5, 3, 1, 5]
After: [1, 3, 3, 5, 5, 5]
Expected: [1, 3, 3, 5, 5, 5]
Result: PASS

--- Already Sorted ---
Before: [1, 2, 3, 4, 5]
After: [1, 2, 3, 4, 5]
Expected: [1, 2, 3, 4, 5]
Result: PASS
```

Figure 7.12. Quick Sort Test Results

```
===== MERGE SORT TEST =====

--- Normal Case ---
Before: [8, 5, 3, 6, 4]
After: [3, 4, 5, 6, 8]
Expected: [3, 4, 5, 6, 8]
Result: PASS

--- Empty Array ---
Before: []
After: []
Expected: []
Result: PASS

--- One Element ---
Before: [11]
After: [11]
Expected: [11]
Result: PASS

--- Duplicates ---
Before: [5, 3, 5, 3, 1, 5]
After: [1, 3, 3, 5, 5, 5]
Expected: [1, 3, 3, 5, 5, 5]
Result: PASS

--- Already Sorted ---
Before: [1, 2, 3, 4, 5]
After: [1, 2, 3, 4, 5]
Expected: [1, 2, 3, 4, 5]
Result: PASS
```

Figure 7.11. Merge Sort Test Results

8. Benchmarking Results and Discussion

8.1. Benchmarking Methodology

The implemented data structures and algorithms were benchmarked to evaluate their practical execution performance for different input sizes. The benchmark uses warm-up runs followed by repeated measured runs, and the average execution time is recorded using `System.nanoTime()`.

For each benchmark, different input sizes are used to observe how execution time changes as the amount of data increases. The measured results are then compared with the theoretical Big O complexity discussed in Section 6.

For fair comparison, the same generated dataset is used for all algorithms at each input size, with a separate copy provided to each sorting algorithm to prevent one algorithm from modifying the input used by another.

8.2. Sorting Algorithm Benchmark Results

Input Size	Bubble Sort	Selection Sort	Insertion Sort	Merge Sort	Quick Sort	Heap Sort
1,000	0.715 ms	0.554 ms	0.536 ms	0.073 ms	0.060 ms	0.116 ms
5,000	13.038 ms	4.689 ms	1.034 ms	0.615 ms	0.228 ms	0.433 ms
10,000	81.003 ms	15.839 ms	3.365 ms	0.705 ms	0.477 ms	0.767 ms

The sorting benchmark was performed using input sizes of 1,000, 5,000, and 10,000 elements. Three warm-up runs and seven measured runs were used, and the average execution time was recorded for each algorithm. The results show that Bubble Sort and Selection Sort required more time as the input size increased, while Merge Sort, Quick Sort, and Heap Sort generally showed better performance. Quick Sort recorded the lowest average execution time among the tested sorting algorithms at all three input sizes.

Figure 8.1. Sorting Algorithm Benchmark Results

```
=====
                        SORTING ALGORITHM BENCHMARK
=====
Warm-up runs      : 3
Measured runs     : 7
Input sizes       : [1000, 5000, 10000]
Random seed       : 12345
=====

Input Size: 1000
-----

Algorithm          Average Time
-----
Bubble Sort        0.715 ms
Selection Sort     0.554 ms
Insertion Sort     0.536 ms
Merge Sort         0.073 ms
Quick Sort         0.060 ms
Heap Sort          0.116 ms

Input Size: 5000
-----

Algorithm          Average Time
-----
Bubble Sort        13.038 ms
Selection Sort     4.689 ms
Insertion Sort     1.034 ms
Merge Sort         0.615 ms
Quick Sort         0.228 ms
Heap Sort          0.433 ms

Input Size: 10000
-----

Algorithm          Average Time
-----
Bubble Sort        81.003 ms
Selection Sort     15.839 ms
Insertion Sort     3.365 ms
Merge Sort         0.705 ms
Quick Sort         0.477 ms
Heap Sort          0.767 ms

=====
                        BENCHMARK COMPLETED
=====
```

8.3. Searching Algorithm Benchmark Results

Input Size	Linear Search	Binary Search
1,000	0.006 ms	0.000 ms
10,000	0.048 ms	0.000 ms
100,000	0.187 ms	0.000 ms

A displayed value of 0.000 ms means that the measured execution time was below the displayed millisecond precision, rather than that the operation took exactly zero time.

The searching benchmark was performed using input sizes of 1,000, 10,000, and 100,000 elements, with the target value placed at the end of the array. Three warm-up runs and seven measured runs were used, and the average execution time was recorded for each algorithm. Linear Search required more time

as the input size increased, while Binary Search remained extremely small at the displayed precision. These results are consistent with the theoretical $O(n)$ complexity of Linear Search and $O(\log n)$ complexity of Binary Search.

Figure 8.2. Searching Algorithm Benchmark Results

```

=====
SEARCHING ALGORITHM BENCHMARK
=====
Warm-up runs      : 3
Measured runs     : 7
Input sizes       : [1000, 10000, 100000]
Search case       : Target at the end of the array
=====

-----
Input Size: 1000
Target: 999
-----

Algorithm          Average Time
-----
Linear Search      0.006 ms
Binary Search      0.000 ms
-----

Input Size: 10000
Target: 9999
-----

Algorithm          Average Time
-----
Linear Search      0.048 ms
Binary Search      0.000 ms
-----

Input Size: 100000
Target: 99999
-----

Algorithm          Average Time
-----
Linear Search      0.187 ms
Binary Search      0.000 ms
=====
BENCHMARK COMPLETED
=====

```

8.4. Data Structure Benchmark Results

Input Size	Dynamic Array addAt(0)	Linked List insertFirst()
1,000	0.695 ms	0.027 ms
5,000	0.741 ms	0.077 ms
10,000	2.662 ms	0.104 ms

Input Size	Stack Push	Stack Pop
1,000	0.016 ms	0.015 ms
10,000	0.091 ms	0.065 ms
100,000	0.302 ms	0.227 ms

Although the total benchmark time increases because more operations are performed, each individual Stack push and pop operation remains $O(1)$.

Input Size	ArrayQueue Enqueue	ArrayQueue Dequeue	LinkedQueue Enqueue	LinkedQueue Dequeue
1,000	0.049 ms	0.026 ms	0.021 ms	0.014 ms
10,000	0.100 ms	0.076 ms	0.126 ms	0.056 ms
100,000	0.471 ms	0.421 ms	0.259 ms	0.167 ms

Although the total benchmark time increases because more operations are performed, each individual Queue enqueue and dequeue operation remains $O(1)$.

Input Size	BST Search
1,000	0.000 ms
10,000	0.000 ms
100,000	0.000 ms

The data structure benchmark compared Dynamic Array and Linked List insertion, Stack operations, Queue operations, and Binary Search Tree searching. Dynamic Array insertion at index 0 required more time as the input size increased because existing elements must be shifted, while Linked List insertion at the beginning remained relatively fast. Stack push and pop operations showed small execution times across increasing input sizes, which is consistent with their $O(1)$ operation complexity. The Queue results also remained relatively small, while the measured times differed between the array-based and linked implementations. BST search was displayed as 0.000 ms at the selected precision, so the result should be interpreted as an execution time below the displayed millisecond resolution rather than zero execution time.

Figure 8.3. Data Structure Benchmark Results

```

=====
DATA STRUCTURE BENCHMARK
=====
Warm-up runs      : 3
Measured runs     : 7
Insertion sizes   : [1000, 5000, 10000]
Operation sizes   : [1000, 10000, 100000]
Random seed       : 12345
=====

DYNAMIC ARRAY vs LINKED LIST INSERTION
=====

Input Size: 1000
-----
Data Structure      Average Time
-----
Dynamic Array addAt(0)  0.695 ms
Linked List insertFirst() 0.027 ms

Input Size: 5000
-----
Data Structure      Average Time
-----
Dynamic Array addAt(0)  0.741 ms
Linked List insertFirst() 0.077 ms

Input Size: 10000
-----
Data Structure      Average Time
-----
Dynamic Array addAt(0)  2.662 ms
Linked List insertFirst() 0.104 ms

=====
STACK BENCHMARK
=====

Input Size: 1000
-----
Operation           Average Time
-----
Stack push()         0.016 ms
Stack pop()           0.015 ms

Input Size: 10000
-----
Operation           Average Time
-----
Stack push()         0.091 ms
Stack pop()           0.065 ms

Input Size: 100000
-----
Operation           Average Time
-----
Stack push()         0.302 ms
Stack pop()           0.227 ms

```

```

=====
QUEUE BENCHMARK
=====

Input Size: 1000
-----
Operation           Average Time
-----
ArrayQueue enqueue()  0.049 ms
ArrayQueue dequeue()  0.026 ms
LinkedList enqueue()   0.021 ms
LinkedList dequeue()  0.014 ms

Input Size: 10000
-----
Operation           Average Time
-----
ArrayQueue enqueue()  0.100 ms
ArrayQueue dequeue()  0.076 ms
LinkedList enqueue()   0.126 ms
LinkedList dequeue()  0.056 ms

Input Size: 100000
-----
Operation           Average Time
-----
ArrayQueue enqueue()  0.471 ms
ArrayQueue dequeue()  0.421 ms
LinkedList enqueue()   0.259 ms
LinkedList dequeue()  0.167 ms

=====
BINARY SEARCH TREE BENCHMARK
=====

Input Size: 1000
Search Target: 234
-----
Operation           Average Time
-----
BST search()         0.000 ms

Input Size: 10000
Search Target: 7068
-----
Operation           Average Time
-----
BST search()         0.000 ms

Input Size: 100000
Search Target: 89285
-----
Operation           Average Time
-----
BST search()         0.000 ms

=====
BENCHMARK COMPLETED
=====

```

8.5. Benchmark Discussion

The benchmark results generally support the theoretical Big O analysis presented in Section 6. The sorting results show that Bubble Sort and Selection Sort become increasingly slower as the input size grows, while Merge Sort, Quick Sort, and Heap Sort scale more efficiently. The searching results also show that Linear Search requires more time as the input size increases, whereas Binary Search remains very fast for sorted data.

For the data structures, Dynamic Array insertion at the beginning required more time than Linked List insertion because elements must be shifted when a new value is added at index 0. Stack and Queue operations remained relatively small across the tested input sizes, which is consistent with their $O(1)$ operation complexity. Overall, the practical benchmark results demonstrate how algorithm and data structure choices affect performance as the amount of data increases.

9. Comparison of Alternatives and Trade-offs

9.1. Data Structure Comparisons

The implemented data structures have different strengths and limitations depending on how data needs to be stored, accessed, and modified. Comparing the alternatives helps identify which structure is more suitable for a particular situation.

Comparison	Main Advantage	Main Disadvantage	Suitable Use
Dynamic Array vs Singly Linked List	Dynamic Array provides fast indexed access, while Linked List supports efficient insertion and deletion at the beginning.	Dynamic Array may require shifting elements; Linked List requires traversal for indexed access.	Dynamic Array for frequent access by index; Linked List for frequent insertions and deletions.
Stack vs Queue	Stack provides LIFO processing, while Queue provides FIFO processing.	Each structure is limited to its specific processing order.	Stack for undo/backtracking; Queue for scheduling and ordered processing.
Array Queue vs Linked Queue	Array Queue provides fixed, compact storage; Linked Queue can grow dynamically.	Array Queue has capacity limits; Linked Queue requires additional node references.	Array Queue when capacity is known; Linked Queue when the size may change.
Binary Search Tree vs Linear Structures	BST can provide faster searching when reasonably balanced.	Performance can degrade to $O(n)$ when unbalanced.	Ordered data requiring searching, insertion, and deletion.

9.2. Algorithm Comparisons

The searching and sorting algorithms also involve trade-offs between simplicity, execution time, memory usage, and input requirements.

Comparison	Better Choice When	Trade-off
Linear Search vs Binary Search	Linear Search for unsorted or small data; Binary Search for large sorted data.	Binary Search requires sorted input, while Linear Search does not.
Bubble/Selection/Insertion vs Merge/Quick/Heap	Simple algorithms for small or nearly sorted data; advanced algorithms for larger datasets.	Simpler algorithms are easier to understand but generally scale poorly.
Merge Sort vs Quick Sort	Merge Sort when consistent $O(n \log n)$ performance is preferred; Quick Sort when low additional memory and good practical speed are preferred.	Merge Sort uses $O(n)$ additional space, while Quick Sort can degrade to $O(n^2)$ in the worst case.
Quick Sort vs Heap Sort	Quick Sort for practical average performance; Heap Sort when consistent $O(n \log n)$ performance and $O(1)$ additional space are important.	Quick Sort has a worst-case $O(n^2)$, while Heap Sort is generally more complex.

Overall, there is no single data structure or algorithm that is best for every situation. The appropriate choice depends on the required operations, input characteristics, execution time, and memory constraints. The benchmark results in Section 8 also show that these theoretical trade-offs can be observed in practical execution performance.

10. Conclusion

This project successfully designed and implemented a Java DSA Toolkit containing fundamental data structures and algorithms from scratch. The implemented data structures include Dynamic Array, Singly Linked List, Stack, Queue, and Binary Search Tree, while the implemented algorithms include Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort.

All implemented components were compiled, executed, and tested in IntelliJ IDEA. Normal cases and relevant edge cases were tested to verify that the data structures and algorithms produced the

expected results. The outputs from these test runs were captured and included in the report as evidence of correctness. The benchmark programs were also executed in IntelliJ IDEA using repeated runs and different input sizes, and the actual benchmark outputs were recorded and presented in the benchmarking section.

The report uses the actual console outputs produced by the Java implementations. The relevant test and benchmark outputs were captured from IntelliJ IDEA and inserted into the document as screenshots/code-output figures to provide clear evidence of the implementation results. These practical results, together with the Big O analysis, demonstrate the differences in efficiency between the implemented alternatives.

Overall, the project provided practical experience in implementing, testing, benchmarking, analyzing, and comparing fundamental data structures and algorithms in Java. It also demonstrated that the choice of an appropriate data structure or algorithm depends on factors such as input size, required operations, execution time, and memory usage.

11. Contribution Statement

Top Keitaro:

Implemented and tested the Singly Linked List and Insertion Sort. Contributed to integration, benchmarking, documentation, and presentation preparation.

Lao David:

Implemented and tested the Binary Search Tree and Binary Search. Contributed to integration, correctness testing, documentation, and presentation preparation.

Sorn Kunwath:

Implemented and tested the Stack and Selection Sort. Contributed to integration, benchmarking, documentation, and presentation preparation.

Chui Vanenteakbot:

Implemented and tested the Queue and Bubble Sort. Contributed to integration, correctness testing, documentation, and presentation preparation.

Keo Ratanak:

Implemented and tested the Dynamic Array and Linear Search. Contributed to integration, benchmarking, documentation, and presentation preparation.

All Members:

Collaboratively implemented and tested Merge Sort, Quick Sort, and Heap Sort and contributed to debugging, integration, benchmarking, report preparation, and final presentation.